coursera

Hands-On-Learning: Fine-tune a Customer Service Model for RetailCorp

Learning Objectives

By the end of this activity, learners will be able to:

- Curate and preprocess domain-specific conversation data for policy-aware LLM fine-tuning.
- Configure LoRA adapters that balance memory footprint with accuracy gains on compliance metrics.
- Evaluate and iterate on fine-tuned models with cost, latency, and policy coverage dashboards.

Description

In this hands-on practice, you are RetailCorp's AI engineering lead responding to a holiday support surge that exposed policy gaps. Your fine-tuning artifacts will feed the enterprise deployment pipeline. You will implement a LoRA-based fine-tuning workflow that lifts policy accuracy while controlling training cost and maintaining latency targets.

Structure of the Activity

Dataset Information/Resources Required

RetailCorp provides 5,000 anonymized customer conversations labeled with intents (returns, sizing, warranty), policy references, and satisfaction ratings. The dataset is stored as JSONL files with token counts under 1,024. Sensitive attributes are redacted; follow the data handling checklist when exporting metrics.

Dataset:

- Jsonl file **(m2_finetune_customer_service_model_retailcorp_conversations. jsonl)**

Tools Used:

- Python 3.10
- PyTorch 2.1+
- Hugging Face Transformers 4.35+
- peft, datasets, accelerate, evaluate, bitsandbytes
- Jupyter Notebook (Lab environment)

Setup Requirements

- Access to starter code file: m2_lab_starter.ipynb in the Coursera lab or your local Jupyter environment
- Install dependencies with: pip install -U transformers peft accelerate datasets bitsandbytes evaluate
- Starter notebook loads datasets, defines evaluation shells, and scaffolds LoRA configuration

Scenario

RetailCorp's holiday support surge exposed critical policy gaps in their customer service responses. As the AI engineering lead, you must respond to executive demands for improved compliance while working within tight GPU budgets. Leadership expects measurable gains in accuracy for policy-specific queries without compromising response latency. The compliance office requires evidence of policy accuracy before any production rollout.

Objective

Implement a LoRA-based fine-tuning workflow that lifts policy accuracy while controlling training cost and maintaining latency targets.

coursera

Instructions for Participants

Activity 1: Curate and Analyze RetailCorp Data

Objective

Policy compliance starts with the right data slices and balanced labels.

Steps to Complete

Step 1: Load the RetailCorp dataset using the provided load_retailcorp_dataset() helper.
Brief guidance: Review the dataset structure, including intents, policy references, and satisfaction ratings.

Step 2: Perform exploratory analysis on intent distribution, policy coverage, and conversation length.
Brief guidance: Use the following code pattern:
from data_prep import load_retailcorp_dataset, stratified_split dataset = load_retailcorp_dataset(DATA_DIR) train_ds, val_ds = stratified_split(dataset, stratify_column='policy_tag', seed=42)

Step 3: Create train/validation splits, ensuring policy categories remain balanced.
Brief guidance: Document any redacted fields so SMEs understand limitations before deployment.

Try It Yourself — Data Profiling:
from analytics import summarize_conversations summary = summarize_conversations(train_ds) # < YOUR CODE HERE >: print out policy coverage and average satisfaction score

Expected Outcome: Stratified splits keep policy categories within ±5% of each other between the training and validation sets.

Tips:

- Which intents currently have the lowest satisfaction ratings?
- How balanced are policy tags across the train/validation splits?
- Filter transcripts for emerging trends (e.g., international returns) and flag gaps for SME review.
- Execute tests/test_split_balance.py to verify distribution parity.
- Expected output: Policy distribution balance check: PASS when categories align.

Activity 2: Configure and Train LoRA Adapters

Objective

Leadership expects measurable accuracy gains without exceeding GPU budget.

Steps to Complete

Step 1: Initialize the base model (e.g., meta-llama/Llama-2-7b-chat-hf) in 8-bit mode.
Brief guidance: Ensure the base model is loaded correctly with quantization enabled.

Step 2: Configure LoRA adapters targeting attention and projection layers using the template configuration cell.
Brief guidance: Use the following code pattern:
from trainer import build_lora_model, train lora_config = { 'r': 16, 'alpha': 32, 'target_modules': ['q_proj', 'v_proj'], 'dropout': 0.05, } model = build_lora_model(base_model_name='meta-llama/Llama-2-7b-chat-hf', lora_config=lora_config) metrics = train(model, train_ds, val_ds, num_epochs=3, lr=2e-4)

Step 3: Train with the provided train_lora.py loop, monitoring loss and policy accuracy metrics.
Brief guidance: Log GPU memory usage (nvidia-smi) to share with operations teams.

Try It Yourself — Adapter Exploration:
from trainer import evaluate_policy_accuracy # < YOUR CODE HERE >: adjust lora_config['r'] and lora_config['alpha'] to test efficiency vs. accuracy # < YOUR CODE HERE >: call evaluate_policy_accuracy(model, val_ds)

Expected Outcome: LoRA adapters cut trainable parameters by >95% while improving policy accuracy by at least 8 points.

Tips:

- How does reducing the LoRA rank impact memory usage and policy accuracy?
- Which validation metrics signal overfitting on compliance language?
- Swap target modules to include feed-forward layers and compare results.
- Run tests/test_lora_config.py to confirm adapter settings meet dimensional constraints.
- Expected output: LoRA configuration valid: True when the config is accepted.

Activity 3: Evaluate, Optimize, and Plan Deployment

Objective

RetailCorp's compliance office requires evidence of policy accuracy before rollout.

Steps to Complete:

Step 1: Generate evaluation dashboards showing accuracy, policy compliance, and latency across validation slices.
Brief guidance: Use the following code pattern:
from evaluation import build_dashboard, measure_latency dashboard = build_dashboard(metrics, val_ds) latency_stats = measure_latency(model, val_ds, target_latency_ms=400)

Step 2: Apply quantization or distillation if latency increases beyond baseline thresholds.
Brief guidance: Monitor latency and apply optimization techniques as needed.

Step 3: Draft a rollout memo summarizing gains, guardrails, and monitoring plans.
Brief guidance: Highlight monitoring hooks (policy-specific dashboards) to reassure compliance officers.

Try It Yourself — Compliance Checklist:
compliance_gaps = [# < YOUR CODE HERE >: add tuples of (policy_tag, failure_rate, remediation_action)]

Expected Outcome: Latency remains within ±10% of baseline after quantization; compliance gaps shrink below 5%.

Tips:

- Which policies still fall below the 95% accuracy goal?
- What guardrails (e.g., retrieval snippets, policy templates) mitigate residual gaps?
- Run A/B evaluation between zero-shot baseline and fine-tuned model for high-risk intents.
- Execute tests/test_policy_dashboard.py to confirm required metrics appear in the dashboard.
- Expected output: Dashboard completeness: PASS when visuals and metrics render.

Compile Your Findings

1. Requirement 1: Data Preparation

Deliverable: Data splits maintain balanced policy tags and satisfaction distributions.

Include:

- Train/validation splits with stratified policy categories
- Intent distribution analysis
- Policy coverage report

- Average satisfaction scores by intent
- Documentation of redacted fields

2. Requirement 2: LoRA Configuration and Training

Deliverable: LoRA adapters configured and trained within GPU/memory budget.

Include:

- LoRA configuration (rank, alpha, target modules, dropout)
- Training metrics (loss curves, policy accuracy progression)
- GPU memory usage logs

- Parameter reduction analysis (>95% reduction)
- Policy accuracy improvement (≥8 percentage points)

3. Requirement 3: Evaluation and Deployment Plan

Deliverable: Validation dashboard shows policy accuracy improved by≥8 percentage points, and a rollout memo drafted with guardrails and a monitoring plan.

Include:

- Evaluation dashboard (accuracy, policy compliance, latency)
- Latency statistics (within ±10% of baseline)

- Compliance gap analysis (below 5%)
- Rollout memo with gains, guardrails, and monitoring plan
- A/B evaluation results (zero-shot vs. fine-tuned)
- Program runs without errors

Summary

This activity provides hands-on experience with parameter-efficient fine-tuning through practical application of LoRA adapters. Participants have developed competency in curating domain-specific data, configuring efficient training workflows, and validating model improvements against compliance metrics, which are directly applicable to enterprise LLM deployment.

Key takeaways:

- Parameter-efficient fine-tuning delivers compliance gains under tight budgets
- Configuring LoRA adapters and validating their impact on domain-specific metrics
- Continuous evaluation and guardrails keep policy accuracy aligned with enterprise expectations

Common Issues & Solutions

Problem: The Bitsandbytes installation fails on the local machine.

Solution: Use the provided GPU runtime or install CPU fallback (pip install bitsandbytes-cudaXXX).

Problem: Policy accuracy declines after quantization.

Solution: Reduce quantization bit width or apply mixed precision on critical layers before re-evaluating.

Resources

Primary Materials:

- **m2_lab_starter.ipynb** - Starter notebook with dataset loaders and LoRA scaffolding
- **m2_finetune_customer_service_model_retailcorp_conversations.jsonl** - RetailCorp anonymized conversations dataset

Additional Resources:

- LoRA in Production: Microsoft's Approach to Efficient Model Adaptation (reading)
- RetailCorp data handling checklist
- Module 2 videos: Fine-tuning Fundamentals and Strategies, Parameter-Efficient Tuning Methods Overview, Hands-on Fine-tuning with LoRA

Solution/Exemplar:

- **m2_lab_solution.ipynb -** Fully commented reference implementation

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.